

SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-03 pairs with TB-M3-03

The Arbiter: Round-Robin with Priority Override

Find and fix the starter arbiter's fairness bug; prove correctness with a cocotb test.

Learning objectives

- Read the starter arbiter.v and identify the one-line bug that violates round-robin fairness.
- Understand the priority-override contract — when it must preempt and when it must NOT starve the elevator.
- Write a cocotb test that forces 1000 back-to-back requests and asserts fairness holds within 2%.
- Earn the 50 XP bonus for an alternate fix that meets timing at 100 MHz on the first try.

Prep checklist

- ☐ SG-M3-02 complete, integration smoke green.
- ☐ TB-M3-03 read, especially the pseudocode in §3.2.
- ☐ You understand the contract: priority_override_req, when 1, grants axiom ON THE NEXT CYCLE. When 0, arbiter round-robins between elevator and axiom.

Step 1 · Read the starter RTL

```
// src/arbiter.v (starter, buggy)
module arbiter (
    input wire      clk,
    input wire      rst_n,
    input wire      priority_req,
    input wire      elev_req_valid,
    output wire     elev_req_ready,
    input wire [3:0] elev_req_payload,
    output reg      grant_elev,
```

```
output reg      grant_axiom,
output wire     queue_nonempty
);

reg last_granted_axiom;
reg pending_request;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        grant_elev      <= 1'b0;
        grant_axiom      <= 1'b0;
        last_granted_axiom <= 1'b0;
        pending_request  <= 1'b0;
    end else begin
        grant_elev <= 1'b0;
        grant_axiom <= 1'b0;

        // Track pending requests
        if (elev_req_valid && !grant_elev) begin
            pending_request <= 1'b1;
        end else if (grant_elev) begin
            pending_request <= 1'b0;
        end

        // Priority mode: AXIOM gets priority, but alternates to be fair
        if (priority_req) begin
            if (last_granted_axiom) begin
                grant_elev      <= 1'b1;
                grant_axiom      <= 1'b0;
                last_granted_axiom <= 1'b0;
            end else begin
                grant_elev      <= 1'b0;
                grant_axiom      <= 1'b1;
            end
        end
    end
end
```

```

        last_granted_axiom <= 1'b1;
    end
end
// Round-robin mode
// BUG LIVES HERE – see SG-M3-03 §Step 2
else if (elev_req_valid || pending_request) begin
    if (last_granted_axiom) begin
        grant_elev          <= 1'b1;
        grant_axiom         <= 1'b0;
        last_granted_axiom <= 1'b0;
    end else begin
        grant_elev          <= 1'b0;
        grant_axiom         <= 1'b1;
        last_granted_axiom <= 1'b10;
    end
end
end
end
end

assign elev_req_ready = grant_elev;
assign queue_nonempty = elev_req_valid || pending_request;

endmodule

```

Step 2 · Find the fairness violation

The priority path writes to `grant_elev` and `grant_axiom` but does NOT update `last_granted_axiom`.
 Consequence: an adversary that wiggles `priority_req` every cycle causes the arbiter to grant axiom every cycle forever — the elevator starves.

This is a classic priority-inversion bug. The fairness contract says: `priority_req` may PREEMPT, but must also COUNT as an axiom grant so that the next non-priority cycle round-robins to elevator.

Step 3 · Write the fairness test

Add to `test/arbiter/test\_fairness.py`:

```
"""Fairness test for arbiter.v

Tests that priority_req does not starve the elevator.
"""

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge, Timer

CLK_PERIOD_NS = 10
RESET_CYCLES = 8

async def start_clock(dut):
    cocotb.start_soon(Clock(dut.clk, CLK_PERIOD_NS, units="ns").start())

async def nominal_reset(dut):
    dut.rst_n.value = 0
    for _ in range(RESET_CYCLES):
        await RisingEdge(dut.clk)
    dut.rst_n.value = 1
    for _ in range(RESET_CYCLES):
        await RisingEdge(dut.clk)

@cocotb.test()
async def test_priority_does_not_starve_elevator(dut):
    """Priority-req adversary: hold priority_req for most cycles;
    elevator must still receive at least one grant within the 2% fairness
    window."""
```

```
dut._log.info("=== Starting fairness test ===")

await start_clock(dut)
await nominal_reset(dut)

# Start with priority high
dut.priority_req.value = 1
dut.elev_req_valid.value = 1
dut.elev_req_payload.value = 0x3

grants_elev = 0
grants_axiom = 0

dut._log.info("Running 1000 cycles with priority_req=1, dropping every 10
cycles...")

for i in range(1000):
    # Drop priority every 10 cycles to let elevator through
    if i % 10 == 5:
        dut.priority_req.value = 0
    else:
        dut.priority_req.value = 1

    await RisingEdge(dut.clk)
    grants_elev += int(dut.grant_elev.value)
    grants_axiom += int(dut.grant_axiom.value)

    if (i + 1) % 100 == 0:
        dut._log.info(f"Cycle {i+1}: elev={grants_elev}, axiom={grants_axiom}")

total = grants_elev + grants_axiom
elev_share = grants_elev / total if total else 0
```

```

dut._log.info(f"=== Results ===")
dut._log.info(f"  grants_elev  = {grants_elev}")
dut._log.info(f"  grants_axiom = {grants_axiom}")
dut._log.info(f"  total          = {total}")
dut._log.info(f"  elev_share   = {elev_share:.3f}")
dut._log.info(f"  required     >= 0.020 (2%)")

assert elev_share >= 0.02, (
    f"elevator starved: elev={grants_elev} axiom={grants_axiom} "
    f"share={elev_share:.3f} < 0.02"
)

dut._log.info(">>> Fairness test PASSED! <<<")

```

```
@cocotb.test()
```

```
async def test_round_robin_works(dut):
```

```
    """With priority_req=0, elevator and AXIOM should alternate."""
```

```
    dut._log.info("=== Starting round-robin test ===")
```

```
    await start_clock(dut)
```

```
    await nominal_reset(dut)
```

```
    dut.priority_req.value = 0
```

```
    dut.elev_req_valid.value = 1
```

```
    dut.elev_req_payload.value = 0x3
```

```
    grants_elev = 0
```

```
    grants_axiom = 0
```

```
    for i in range(100):
```

```
        await RisingEdge(dut.clk)
```

```

    grants_elev += int(dut.grant_elev.value)
    grants_axiom += int(dut.grant_axiom.value)

total = grants_elev + grants_axiom
elev_share = grants_elev / total if total else 0

dut._log.info(f"=== Results ===")
dut._log.info(f"  grants_elev = {grants_elev}")
dut._log.info(f"  grants_axiom = {grants_axiom}")
dut._log.info(f"  elev_share   = {elev_share:.3f}")

assert 0.4 < elev_share < 0.6, (
    f"Round-robin not working: elev_share={elev_share:.3f}, expected ~0.5"
)

dut._log.info(">>> Round-robin test PASSED! <<<")

```

Run it:

```

cd test/arbiter
make clean
make MODULE=test_fairness

```

Step 4 · Apply the minimum fix

One line. In the priority branch:

```

else if (elev_req_valid || pending_request) begin
    if (last_granted_axiom) begin
        grant_elev      <= 1'b1;
        grant_axiom     <= 1'b0;
        last_granted_axiom <= 1'b0;
    end else begin
        grant_elev      <= 1'b0;
        grant_axiom     <= 1'b1;
    end
end

```

```

        last_granted_axiom <= 1'b1;    ←---- Fixed
    end

```

Re-run the fairness test. It should now pass within 2% elevator share. If your share drops below 2% even with the fix, your reset path is not clearing last_cd test/arbiter granted_axiom — check the `if (!rst\_n)` branch.

```

make clean
make MODULE=test_fairness

```

```

# Expected (with the bug):
#   test_priority_does_not_starve_elevator: PASS

```

```

Grants: elev=499, axiom=500
elev_share: 0.499 (≥ 0.020 requirement)
test_round_robin_works: PASSED
Grants: elev=50, axiom=50
elev_share: 0.500

```

Step 5 · Bonus — 50 XP for a parameterised round-robin

The minimum fix covers 2-way arbitration. The bonus path generalises to N-way, readable in one line of Verilog for any N:

```

parameter N = 2;
reg [N-1:0] grant_onehot, req_onehot;

// rotate a one-hot pointer right, gate it by pending requests
wire [N-1:0] next_grant = rotate_right(grant_onehot) & req_onehot;

```

The bonus 50 XP is awarded for: parameterised arbiter that works at N=2 and N=4, fairness test holds, and clocked timing meets 100 MHz on the first LibreLane try. You do NOT need to instantiate N=4 in top.v — the grader will sweep it on a side testbench.

Step 6 · Regenerate metrics

```

./scripts/run-librelane.sh --tag arbiter-fix

cat ./librelane/runs/$(ls -t ./librelane/runs/ | head
-1)/55-openroad-stapostpnr/summary.rpt

```

```

Setup WNS (TT Corner):  5.4058 ns

```



```
Setup WNS (Overall):    3.9761 ns
Hold WNS (Overall):    0.1620 ns
Violations:            0
```

```
# WNS (worst negative slack) positive or zero at CLOCK_PERIOD=10ns
```

```
cat ./librelane/runs/$(ls -t ./librelane/runs/ | head
-1)/55-openroad-stapostpnr/nom_tt_025C_1v80/ws.max.rpt
```

```
=====
Worst Slack (Setup)
```

```
=====
nom_tt_025C_1v80: 5.405828382386134
```

Troubleshooting

Fairness test passes but timing fails	Your fix introduced a long combinational path. Register the next_grant. Don't collapse it into assign.
`last_granted_axiom` shown as X in waves	You missed it in the `if (!rst_n)` branch. Add `last_granted_axiom <= 1'b0;`.
elev_share > 0.98	You may have broken priority_req — it's now being ignored. Re-check the priority branch still sets grant_axiom=1.

Reflection

- Which view helped you find the bug first — reading the code, reading the waveform, or reading the failing test output? Note it for your escape log.
- Write a one-sentence summary of the fairness contract in your own words. If you can't, re-read TB-M3-03 §3.2.